

could just show a game introduction with a Start button. When the user clicks the Start button, the game will execute an assignment statement that changes the value of the `state` variable to transition into the Play state:

```
state = STATE_PLAY
```

Once that line runs, only the code in the first `elif` runs and completely different code will execute—the code to show and react to the Play state.

Similarly, whenever and however the program reaches the ending condition for the game, it will execute the following line to transition to the End state:

```
state = STATE_END
```

From then on, every time the program goes around the `while` loop, the code of the second `elif` branch will run.

In summary, a state machine has a set of states, one variable to keep track of which state the program is in, and a set of events that cause the program to transition from one state to another. Since there is a single variable that keeps track of the state, the program can be in only one of the states at any time. Different actions that the user takes (clicking a button, pressing a key, dragging an item, and so on) or other events (such as a timer running out) can cause the program to transition from one state to another. Depending on which state it is in, the program may listen for different events and will typically execute different code.

A pygame Example with a State Machine

Next, we'll build a Rock, Paper, Scissors game that uses a state machine. The user chooses rock, paper, or scissors; then the computer randomly chooses among the three. If the person and computer choose the same item, it's a tie. Otherwise, one point is awarded to the player or the computer, according to the following rules:

- Rock crushes scissors.
- Scissors cuts paper.